

FTRIA-108 — Runtime Alignment Operators

Part II — Runtime Invariant Algebra

Sizhe Tan

with chatGPT (OpenAI)

2026-07-16

Repository: <https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA>

Abstract

Runtime Invariant Discovery begins before a Runtime Invariant has been identified.

Its first challenge is to establish plausible structural correspondences among multiple observations, representations, or runtime structures.

This paper introduces **Runtime Alignment Operators (RAOs)** as a foundational operator family within Runtime Invariant Discovery Algebra. A Runtime Alignment Operator does not yet determine the final Runtime Invariant.

Instead, it generates, evaluates, and organizes candidate correspondences that make later consensus formation, invariant construction, and validation possible.

The Bucket Tree of Permutation (BTP) provides one important implementation of this operator family.

Across ImageStarmap, GraphStarmap, SequenceStarmap, Differential Trees, Universal Task Networks, and related Structural Intelligence systems, BTP demonstrates that one reusable alignment engine can support many domains when supplied with domain-specific node-matching and constraint APIs.

Runtime Alignment therefore represents both a practical engineering mechanism and a general algebraic stage in the construction of Runtime Intelligence.

1. Existing Algorithmic Experience

Structural Intelligence has repeatedly encountered the same computational problem.

Given multiple structures whose elements are not pre-aligned, how can the system discover the most plausible structural correspondence among them?

Examples include

- image structures,
- graphs,
- sequences,
- code structures,
- task structures,
- semantic structures.

The individual domains differ.

However, the underlying problem remains similar:

Find the structural arrangement that permits the strongest admissible integration of multiple runtime observations.

ImageStarmap,

GraphStarmap,

SequenceStarmap,

and related algorithms address this problem through Unaligned AND and the Bucket Tree of Permutation.

2. Traditional Interpretation

Traditionally, these methods may be described using terms such as

- permutation search,
- node matching,
- graph alignment,
- sequence correspondence,
- structural integration,
- candidate matching.

These descriptions are operationally accurate.

However, they tend to emphasize implementation details rather than the common Runtime Function.

FTRIA proposes a higher-level interpretation.

3. Runtime Invariant Interpretation

Within Runtime Invariant Discovery Algebra, these methods perform

Runtime Alignment.

A Runtime Alignment Operator establishes candidate correspondences among runtime elements before a stable Runtime Invariant has been formed.

Conceptually,

Multiple Runtime Observations



Candidate Correspondences



Runtime Alignment Operator



Aligned Structural Hypotheses

The output is not yet a certified Runtime Invariant.

It is a set of candidate structural organizations upon which later Discovery stages can operate.

4. Definition of a Runtime Alignment Operator

A **Runtime Alignment Operator (RAO)** is an operator that searches for structurally admissible correspondences among multiple runtime observations or representations.

Its responsibilities may include

- node matching,
- permutation generation,
- candidate alignment construction,
- partial correspondence preservation,
- structural constraint propagation,
- candidate pruning,
- alignment ranking.

A Runtime Alignment Operator therefore converts unaligned observations into candidate structural hypotheses.

5. Bucket Tree of Permutation as an Alignment Engine

The Bucket Tree of Permutation (BTP) provides a reusable implementation of Runtime Alignment.

Its central role is to explore candidate structural arrangements while controlling the combinatorial growth of possible permutations.

Conceptually,

Unaligned Structures



Bucket Tree of Permutation



Candidate Structural Alignments



Vote and Trim



Common Concept Core

BTP is important not merely because it searches permutations. Its deeper value lies in its ability to organize permutation search into a reusable structural engine.

6. BTP as a Unified Node-Matching Architecture

One of BTP's strongest engineering properties is the separation between

- the general alignment engine, and
- the domain-specific matching logic.

The BTP engine manages

- candidate organization,
- permutation exploration,
- branch expansion,
- pruning,
- structural accumulation.

The domain implementation supplies APIs for

- node compatibility,
- match scoring,
- structural constraints,
- link constraints,
- domain-specific admissibility.

Conceptually,

General BTP Engine



Domain-Specific APIs



Image Alignment

Graph Alignment

Sequence Alignment

Code Alignment

Task Alignment

This division enables one Runtime Alignment architecture to support many structural domains.

7. Plugin Constraints and Extensibility

Runtime Alignment rarely depends on node similarity alone.

Real structures contain

- links,
- ordering,
- dependency relations,
- direction,
- hierarchy,
- semantic conditions,
- partial constraints.

BTP naturally supports additional plugin logic that constrains or evaluates candidate alignments.

Examples include

- graph-link compatibility,
- sequence-order constraints,
- dependency consistency,
- parent-child constraints,
- runtime feasibility rules.

This extensibility is essential.

It allows Runtime Alignment to evolve from simple node matching toward richer Runtime Structural Organization.

8. Alignment Is Not Yet Consensus

Runtime Alignment generates candidate structural hypotheses.

It does not by itself determine which structure is invariant.

The subsequent Discovery stages remain necessary.
Runtime Alignment



Vote and Trim



Structural Consensus



Runtime Invariant Formation



Validation

This distinction is important.

Alignment answers:

Which structures may correspond?

Consensus answers:

Which correspondences remain stable across candidates and observations?

Runtime Invariant Formation then converts that stability into a reusable structural object.

9. Relationship to Common Concept Core

Common Concept Core depends upon successful Runtime Alignment.

Without alignment,

shared structural elements may remain hidden because their positions, orders, labels, or representations differ.

Runtime Alignment makes Unaligned AND possible.

Vote and Trim then identify the stable common structure.

Conceptually,

Unaligned AND



Runtime Alignment



Structural Consensus



Common Concept Core



Runtime Invariant

CCC is therefore not produced by matching alone.
It emerges from alignment followed by consensus and validation.

10. Beyond Immediate Recognition

Runtime Alignment is often used for recognition and scoring.
However, recognition is only one application.

The same operator family can support the construction of

- Differential Trees,
- Universal Task Networks,
- Calling Graphs,
- Runtime Graphs,
- structural prototypes,
- reusable semantic organizations.

In these cases, Runtime Alignment contributes to structure construction rather than immediate classification.

This broader role leads naturally toward Runtime Structural Organization.

11. Relationship to Modern AI

Many modern AI mechanisms perform functions related to Runtime Alignment.

Examples include

Transformer-based AI

- token-to-token Attention,
- contextual correspondence,
- cross-modal alignment,
- retrieval alignment.

Vision AI

- feature correspondence,
- object-part alignment,
- multi-scale feature alignment.

Graph AI

- node correspondence,
- graph matching,
- structural embedding alignment.

Multimodal AI

- text-image alignment,
- audio-text alignment,
- cross-representation mapping.

These systems use different implementations, but they share the Runtime Function of establishing context-dependent structural correspondence.

FTRIA does not claim that these mechanisms are identical to BTP. Instead, it places them within the same broad Runtime Alignment family.

12. Engineering Implications

Treating Runtime Alignment as an explicit operator family provides several advantages.

It enables

- reuse of one alignment engine across domains,
- plugin-based constraint extension,
- separation of alignment from consensus and validation,
- independent optimization of search and matching APIs,
- integration of symbolic and neural alignment methods,
- migration from recognition systems toward structure-building systems.

Most importantly,

it prevents Runtime Invariant Discovery from being reduced to a single similarity score.

Runtime Alignment is a structural search problem, not merely a metric-comparison problem.

13. From Alignment to Structural Organization

Runtime Alignment establishes correspondence.

However, intelligent systems frequently need more.

They must also construct

- groups,
- hierarchies,

- networks,
- dependencies,
- execution pathways,
- reusable runtime structures.

Therefore,

Runtime Alignment should be understood as a foundational specialization of a broader Runtime Structural Organization capability.

Conceptually,

Matching



Runtime Alignment



Runtime Structural Organization



Runtime Invariant



Runtime Triggering

This progression is developed further in FTRIA-117 and FTRIA-191.

Key Takeaways

- Runtime Alignment is the first structural stage of Runtime Invariant Discovery.
- A Runtime Alignment Operator generates candidate correspondences rather than final Runtime Invariants.
- Bucket Tree of Permutation provides a reusable implementation of Runtime Alignment.
- BTP separates the general permutation and organization engine from domain-specific node-matching and constraint APIs.
- Vote and Trim convert candidate alignments into Structural Consensus.
- CCC formation and validation occur after Alignment.
- Runtime Alignment supports not only recognition, but also Differential

Tree, UTN, Calling Graph, and Runtime Structure construction.

- Runtime Alignment is an important specialization within the broader Runtime Structural Organization family.

Related FTRIA Documents

Discovery Algebra

- FTRIA-101 — Runtime Invariant Discovery Algebra
- FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant Discovery
- FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding
- FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant Discovery
- FTRIA-109 — Runtime Invariant Discovery Pipeline

Structural Organization

- FTRIA-117 — Runtime Structural Organization Operators
- FTRIA-118 — Operator Economy of Intelligence

Future Directions

- FTRIA-191 — From Runtime Alignment to Runtime Structural Organization
- FTRIA-192 — The Runtime Organization Ladder as an AI Evolution Map